Multimodal Systems - Project Report

Azzurra Suffia
5341007

March 2, 2026

Contents

1 Introduction **3**

2 Sensor devices choice **3**

3 System Design **6**
3.1 Critical aspects 7

4 Python Prototype **8**

Note: The development of this project and the preparation of this report were assisted by a large language model (ChatGPT 5). The LLM was used for tasks such as final editing, grammar checking, and code refactoring. The project logo was created using NanoBanana.

1 Introduction

This project aims to develop a prototype of an interactive system for an art installation. When a visitor enters the room, two digital walls appear and dynamically adjust to the sides of the visitor's bounding box. The visitor can expand this rectangle to gradually reveal the background of the room. When the visitor's energy falls below a predefined threshold, the visitor disappears and the two walls collapse toward the center, merging in the middle.
The following sections present the choice of sensing device for the application, the overall system design, its critical aspects, and the main components of the prototype.

2 Sensor devices choice

To begin with, it is necessary to identify the sensor devices to use in system development.
To address this, we must first determine what type of information the system needs to capture in order to fulfill its intended functionalities, and how such information can be provided by the user and/or the environment.

- the left and right x-coordinates of the bounding rectangle;
- a measure of energy (thus requiring blobs for motion index estimation or landmarks for computing kinetic energy or quantity of motion).

Since the application is designed for an art installation, a **controlled environment** can reasonably be assumed. However, no control can be exercised over visitors' clothing. Furthermore, the system is required to operate in **real time**.

Device	**Pros**	**Cons**
IMU (Inertial Measurement Units)	• Low cost • Wearable or embedded in portable devices and lightweight • Not affected by lighting conditions and occlusions	• Users must wear sensors • Position estimation requires integration of acceleration (drift over time)
Optical Motion Capture	• Very high precision (millimeter-level) • Reliable 3D tracking for detailed motion analysis	• Expensive • Users must wear reflective markers • Requires controlled lighting conditions • Complex installation and calibration

Range-Imaging Device (Depth Camera)	• Good precision • No wearable sensors or markers required • Native body segmentation	• Sensitive to strong ambient light (IR interference) • Limited operating range • Occlusion issues
Standard RGB Cameras	• Low cost • Widely available • No wearable sensors or markers required	• Requires computer vision algorithms for pose detection and estimation • Lower precision compared to marker-based motion capture systems • Occlusion issues • Sensitive to lighting variations
Magnetic Motion Capture	• Low latency • Works without cameras • Not affected by lighting conditions and occlusions	• Users must wear sensors • Sensitive to magnetic interference • Limited tracking range
Mechanical Motion Capture	• High accuracy in joint angle measurement • Stable over long sessions • Not affected by lighting conditions and occlusions	• Users must wear an exoskeleton • Restricts natural movement • May lack an absolute global reference frame

Table 1: Comparison of Sensing Devices

The best choices are **Depth Cameras** and **Standard RGB Cameras**. Indeed, both are non-intrusive, since they do not require special clothing or markers, operate in controlled indoor environments, and can provide both the bounding rectangle and energy/motion estimation in real time. However, RGB Cameras do not natively provide tools for pose detection and tracking; hence, a further processing step is required. Given that only standard RGB cameras are available, the prototype will be developed using this capturing technique.

Among the possible approaches for capturing movement from a video stream, two main options were considered: blob-based motion detection and landmark-based pose estimation. Table 2 summarizes the pros and cons of each approach. For this application, the landmark-based approach was implemented. Medi-

aPipe was chosen as the pose estimation tool due to its lightweight design and real-time performance, even on standard CPUs.

Approach	**Pros**	**Cons**
Blob Detection + Motion Index	• Fast execution • Low computational cost • Minimal hardware requirements • Simple implementation	• Sensitivity to visitors' clothing colors • Sensitivity to lighting conditions • Dependence on blob detection quality • Ineffectiveness of Motion Index for movements toward or away from the camera
Landmark Detection + Kinetic Energy	• Robustness to visitors' clothing colors and background variations • Reliability of body bounding rectangle estimation • Accuracy in detecting body expansion and depth movement	• Higher computational requirements • Requirement for more powerful hardware • Sensitivity to body occlusions • Complexity of implementation

Table 2: Comparison between Blob-Based and Landmark-Based Approaches

3 System Design

In the context of the layered framework by Camurri et al. [1], a direct mapping strategy is sufficient. There is no need to reach the concepts and structures layer, as all computations and actions can be performed at the low-level features and physical signals layers.

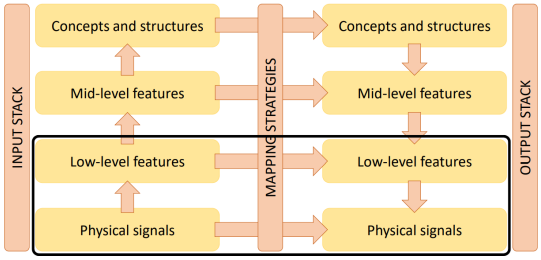

Figure 1: Layered framework for multimodal interaction, as introduced by Camurri et al. [1].

Building on this layered framework, the system pipeline for the current application can be represented as a sequence of processing steps, from recognition to output rendering. Figure 2 provides an overview of this pipeline, illustrating how raw physical signals are transformed through kinematic estimation, feature extraction, mapping, and synthesis to generate the final interactive output.

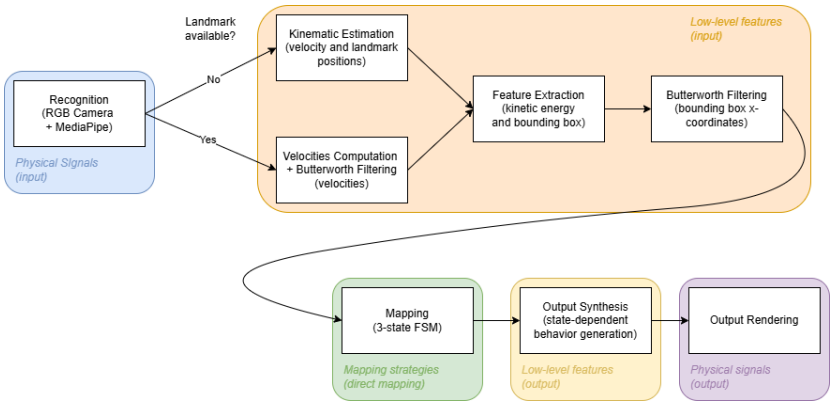

Figure 2: Overview of the System Pipeline.

The mapping phase of the pipeline is implemented using a Finite State Machine (FSM). Figure 3 illustrates the FSM, showing the different states and the conditions that trigger transitions between them.

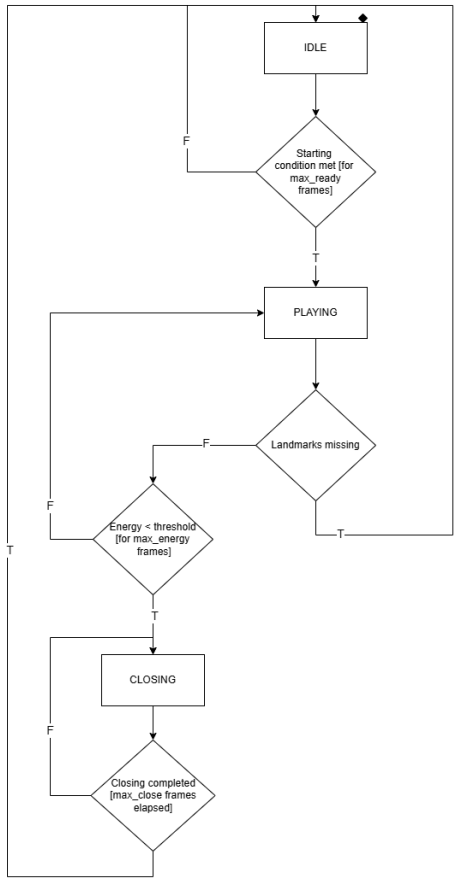

Figure 3: Finite State Machine used for the mapping phase.

3.1 Critical aspects

The system is an interactive application, which means that **system responsiveness** to the user's input is crucial. Visitors should perceive the interaction as smooth and coherent with their actions; otherwise, the application could cause frustration. For this reason, state transitions are never based on features computed from a single frame, but rather on a window of frames defined by the constants `MAX_READY`, `MAX_CLOSE`, and `MAX_ENERGY`.

Moreover, the computation of kinetic energy is critical. Predicted landmarks inherently jitter due to image noise, body occlusions, and model uncertainty. Such variations make velocity values unstable, and differentiation further degrades the signal-to-noise ratio. To mitigate this, a Butterworth filter is applied to velocities before squaring in the kinetic energy computation.

Finally, MediaPipe models, especially the Lite versions that prioritize speed over accuracy, may occasionally skip landmark predictions if a result is not delivered on time — particularly when the movement is toward or away from the camera. To address this, the `BodyEstimator` class replaces missing velocities and landmarks by applying a decay to the previous velocities and estimating landmark positions from them:

$$\mathbf{v}_t = \alpha\, \mathbf{v}_{t-1} \tag{1}$$

$$\mathbf{p}_t = \mathbf{p}_{t-1} + \mathbf{v}_t\, \Delta t \tag{2}$$

This estimation prevents drops in the computed kinetic energy and maintains the perception of continuous interaction.

4 Python Prototype

The camera used in the prototype operates at 30 FPS with a resolution of 1920×1080 pixels.

The main application script is `main.py`. Several support classes were implemented:

- `BodyEstimator`, which estimates velocities and positions when landmarks are missing;
- `KEProcessor`, for computing kinetic energy;
- `InteractionFSM`, which manages state transitions;
- `ButterworthMultichannel`, for filtering velocities and bounding box x-coordinates.

Additional utility functions include:

- `get_bounding_rectangle`, to obtain bounding rectangle coordinates;
- `is_user_ready`, which checks whether the user is visible and standing near the center of the screen;
- `compute_wall_positions`, for computing frame-by-frame wall positions during the closing phase.

Finally, several drawing methods were implemented to render walls and other graphical elements.

References

[1] Alessandro Camurri, Barbara Mazzarino, Mauro Ricchetti, Rene Timmers, and Giancarlo Volpe. Multimodal analysis of expressive gesture in music and dance performances. In Alessandro Camurri and Giancarlo Volpe, editors, *Gesture-based Communication in Human-Computer Interaction*, volume 2915 of *Lecture Notes in Computer Science (LNCS)*, pages 20–39. Springer, 2004.